



PayScan: Detection and Security Analysis of Payment Libraries in Android Apps

Fadi Mohsen¹ · Manar Alohalay² · Usman Rauf³ · Dominic Therattil¹ · Loran Oosterhaven¹

Accepted: 20 July 2025 / Published online: 31 July 2025
© The Author(s) 2025

Abstract

Third-party payment libraries (TPLs) are widely used in Android applications to facilitate in-app transactions, yet their security implications remain largely underexplored. In this paper, we present a novel approach for automated detection and security analysis of payment libraries in Android applications. Our tool, PayScan, employs byte-pattern analysis and heuristic scanning techniques to identify TPLs and then it assesses their security posture. Additionally, the tool integrates three independent security scanners. We analyzed a dataset of 10,553 Android applications, detecting 18 payment libraries and evaluating their security and privacy risks. Our findings indicate that 71.7% of applications use outdated payment libraries, with some SDK versions being over four years old. Additionally, we identified 397 private key leaks across 212 applications. The security scanners detected over 20,000 vulnerabilities, including critical issues such as SSL misconfigurations, WebView XSS, and weak cryptographic implementations. We compare our detection approach against LibScout and LibRadar, demonstrating its practical performance in detecting payment libraries, including in obfuscated applications. This study reveals important security risks in mobile payment ecosystems and emphasizes the value of continued monitoring of third-party payment libraries. The proposed tool offers a scalable solution for detection and analysis, providing practical utility for researchers, developers, and auditors focused on financial application security.

Keywords Mobile · Payment · Android · Vulnerabilities · Security

1 Introduction

Mobile applications have become integral to everyday life in the rapidly evolving digital landscape. The global smartphone user base is projected to grow by 23.1% between 2022 and 2028, reaching 6.2 billion users [8]. With this expansion in user base, mobile wallet transactions are expected

to reach \$9 trillion in monetary value by the end of 2025 [26]. This growth has led to an increased reliance on third-party libraries (TPLs). A prior study revealed that, in certain instances an application incorporated over 30 distinct TPLs [16]. This prevalence highlights their significance, as TPLs empower developers to integrate crucial functionalities such as analytics, payment processing, and advertisements into their applications. Conversely, end users face heightened privacy risks due to outdated TPLs, with approximately 70% of these libraries being outdated, amplifying concerns as older software components are shown to carry a vulnerability rate three times higher than their updated equivalents [10] [7].

Many mobile applications collect user data, and vulnerabilities within these apps or their embedded TPLs can expose sensitive information to cybercriminals. A study of mobile money applications revealed that 40% of high-reputation organizations neglect basic vulnerability scans, highlighting a critical gap in the security practices of commercial software [7]. Issues such as poor SSL/TLS configurations, inadequate user authentication, and unnecessary data collection further exacerbate these risks [11, 27, 28]. Additionally, research

✉ Fadi Mohsen
f.f.m.mohsen@rug.nl
Manar Alohalay
mfalohaly@pnu.edu.sa
Usman Rauf
urauf@mercy.edu

¹ Bernoulli Institute for Mathematics, Computer Science and Artificial Intelligence, University of Groningen, 9747 AG Groningen, Netherlands

² Information Systems Department, Princess Nourah Bint Abdulrahman University, Riyadh, Saudi Arabia

³ Department of Mathematics and Computer Science, Mercy University, Dobbs Ferry, NY, USA

has shown that TPLs in Android applications, particularly in third-party payment systems, face various security challenges [14, 15].

While significant research has focused on identifying ad libraries, there is limited work on detecting and analyzing third-party payment libraries specifically. Addressing this gap, our study aims to shed light on the security and privacy state of payment libraries in Android applications. We developed a solution to identify these libraries within application binaries, scanned the identified apps for vulnerabilities, and categorized the findings related to payment libraries.

Existing research has primarily targeted popular libraries using large datasets for training detection models [3, 17, 20, 38]. However, our approach leverages heuristic pattern scanning and invocation tracing to identify third-party payment libraries more effectively. In addition to vulnerability detection, our analysis includes privacy leak assessments, addressing a critical aspect often overlooked in prior studies [9, 13, 22, 25]. Beyond detection, identifying TPLs is crucial for a wide range of security-related research areas, including repackaging detection, license violation analysis, and the detection of suspicious in-app advertisements [37].

By focusing on third-party payment libraries, this work aims to provide valuable insights into their security and privacy implications, contributing to more robust mobile application security practices.

Despite growing interest in third-party SDK analysis, little attention has been paid to security issues specific to mobile payment libraries. Our work fills this gap with the following novel contributions: (i) a scalable tool (PayScan) that combines manifest-based heuristics and byte-pattern detection to identify 18 different payment SDKs, including in obfuscated code; (ii) correlation of vulnerability reports from multiple static analysis tools, enabling attribution of security issues to specific libraries; and (iii) a comprehensive comparative evaluation against existing tools (LibScout, LibRadar) across detection accuracy, runtime efficiency, and obfuscation resilience. To our knowledge, this is the first large-scale security study focused exclusively on mobile payment libraries.

The remainder of this paper is structured as follows: We begin in Section 2 with an overview of relevant background topics. In Section 3 we provide an overview of related works. Section 4 presents our methodology, encompassing the identification and security analysis of payment libraries. Next, in Section 5, we delve into the results of our evaluation and comparison studies. Finally, we conclude the paper in Section 6.

2 Background

In this section, we provide brief background information on the Android application package's (APK) structure, mobile third-party payment, Android vulnerability scanners, TPL detection and obfuscation.

2.1 Android Application Package (APK)

The *.apk* that runs on an Android OS-based platform consists of a collection of files that make up an Android application. At the core is the Manifest, an XML file located at the root of the application hierarchy. It describes essential information about the app's structure and metadata of the app and declares the necessary permissions to access data from the system or other apps. The Resources directory contains assets such as drawables, layouts, and menus, organized to support different screen orientations, OS versions, and languages. Compiled resources provide information about pre-compiled assets, including their names, configurations, types, and paths. The Assets directory houses application resources like sounds, text, images, videos, and audio in a developer-controlled hierarchy, retrievable via *AssetManager*. Native libraries contain the machine code for each supported processor. The Signature directory holds verification data for each file within the APK, ensuring integrity by requiring re-signing after any modification. Finally, the Dalvik bytecode includes the *class.dex* files, which results from compiling the Java or Kotlin source code into bytecode.

2.2 Vulnerability Scanners in Android

Vulnerability scanners are tools designed to identify weaknesses in a system that could lead to security or privacy risks. These tools can be broadly categorized into static scanning and dynamic scanning methods. Static scanning entails analyzing the source code or application binaries without executing them, focusing on identifying vulnerabilities early in the development lifecycle. Dynamic scanning, on the other hand, tests the system during runtime, simulating real-world attacks to uncover vulnerabilities that only manifest during execution. Vulnerability scanners can also be classified by their target environments, such as network, web, database, cloud, and source code scanners. Unlike other vulnerability scanning tools, source code scanners aim at early detection and correction of vulnerabilities, making them particularly valuable in the software development process. In Android, source scanning and security data flow analysis tools work by decompiling and analyzing files inside the Android Application Package (*.apk*) extension [23]. In this work, we built on top of Androwarn [34], FlowDroid [33], and SuperAnalyzer [35] to identify third-party payment libraries and scan them for security and privacy vulnerabilities.

2.3 Obfuscation Techniques

The objective of code obfuscation is to make reverse engineering more challenging while maintaining functionality [1]. There are various kinds of obfuscation techniques such as control flow randomization, identifier renaming, string encryption, API hiding, and class encryption. The techniques vary in complexity and reversibility [19]. The utilization of code obfuscation techniques in Android apps is a common practice. As such, any solution entailing scanning APK files must consider obfuscation. Below we discuss briefly some of these techniques as we will refer to some of them in our discussion of our tool. We start with the Control Flow Randomization, which obscures the control flow graph (CFG) by altering its structure without changing execution. This includes adding spurious branches or flattening existing ones, making reverse engineering more difficult [19, 36]. Identifier Renaming changes the names of variables, classes, and methods to non-descriptive terms (e.g., "a1" or "x23"), removing semantic meaning and obscuring functionality [19, 36]. String Encryption safeguards sensitive data like API keys and passwords by encrypting strings in the source code. Decryption occurs only at runtime, protecting the data from static analysis [19, 36]. API Hiding conceals application programming interfaces (APIs) by dynamically referencing them by creating address tables or using secret index variables, complicating static and dynamic analysis [6]. Finally, the Class Encryption, which transforms class definitions into an encrypted format, is decrypted at runtime. This ensures classes remain unreadable in their static form [12].

3 Related Work

Our discussion of related work is divided into two main aspects: (1) the identification of payment libraries in Android applications and (2) the security analysis of Android applications embedding such libraries. Research on third-party library (TPL) identification in Android applications has focused primarily on two aspects: detection techniques and security/privacy analysis. Despite significant advancements, current approaches still face notable limitations [37]. Most existing tools exhibit high precision but low recall, meaning they can accurately detect known TPLs but struggle to identify new or less common ones. Additionally, many techniques lack robust handling of code obfuscation and optimization, which are common in real-world applications and significantly hinder detection accuracy.

Existing studies vary in their assumptions about application developers (malicious vs. benign) and the types of embedded libraries they target (advertising, analytics, or payment-related). Several works, such as those by Ma et al. [20], Li et al. [17], Zhang et al. [38], and Backes et al. [3], rely

on supervised learning techniques trained on large datasets of labeled applications. While effective for detecting widely used libraries, these approaches are less suitable for our study because access to labeled datasets of payment libraries is limited, and their popularity varies widely. Similarly, detection techniques proposed by Liu et al. [18] and Narayanan et al. [24] focus primarily on advertising libraries, making them unsuitable for our payment library identification needs.

In addition to library detection, security researchers have explored vulnerabilities in third-party components. Moon-samy and Batten [22] analyzed data leaks in ad-embedded applications, relying on DroidBox [5] to trace leaks to third-party entities. However, their approach depends on a single vulnerability analysis tool and focuses on namespace-based identification, which is less adaptable to obfuscated or optimized code. In contrast, our approach first detects the payment library and then parses vulnerability reports to isolate those affecting the detected libraries. Other works, such as those by Short and Li [25], Demetriou et al. [9], and Grace et al. [13], primarily examine security vulnerabilities in ad libraries, with an emphasis on sensitive data leaks. While valuable, this line of research does not address payment libraries, which pose distinct security challenges due to their role in financial transactions.

In summary, while prior work has made significant strides in TPL detection and security analysis, no existing approach is well-suited for identifying and analyzing security risks, specifically within payment libraries. Our work addresses this gap by focusing on robust detection techniques tailored to payment libraries and analyzing their associated security vulnerabilities.

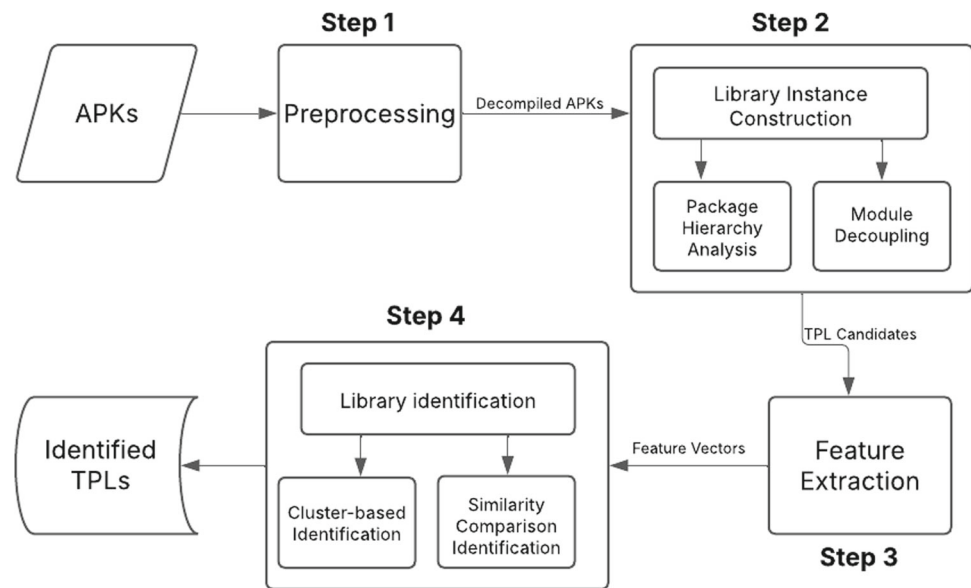
4 Methodology

To investigate the security risks of third-party payment libraries in Android applications, we developed *PayScan*, a tool designed to detect and analyze embedded payment SDKs. *PayScan* operates by first detecting the presence of payment libraries within the binaries of an application. Then, it scans the binaries for security vulnerabilities associated with the detected libraries. In this section, we start with an overview of the TPL detection as per the literature, then we move into describing *PayScan* in depth.

4.1 TPL Detection: General Methodology

Our analysis of existing approaches to TPL detections [36], resulted in identifying two pioneering tools, namely LibScout [4] and LibRadar [21]. LibScout is recognized for its high accuracy, while LibRadar stands out for its exceptional efficiency. Additionally, both tools are open-source and well-documented, making them ideal candidates for our

Fig. 1 Overview of the third-party library (TPL) detection pipeline



comparative study. This selection guarantees a comprehensive assessment of our tool's performance in comparison to the leading industry alternatives. Building on insights from Zhan et al. [36], LibScout [4], and LibRadar [21], we outline a general processing pipeline for detecting third-party libraries (TPLs) as illustrated in Figure 1. This pipeline consists of four key stages: preprocessing, library instance construction, feature extraction, and library identification.

In the **preprocessing stage**, Android applications in APK format are decompiled using reverse-engineering tools. This transformation converts the application data into an intermediate representation, typically in the form of bytecode, which serves as the foundation for subsequent analysis. By converting the application into a more analyzable format, this step ensures that relevant structural and behavioral features can be effectively extracted.

Following preprocessing, the **library instance construction stage** involves identifying and isolating potential TPL candidates from the decompiled application code. This can be achieved using package hierarchy analysis or module decoupling algorithms. Package hierarchy analysis detects independent Java packages within the application, where each independent package is considered a potential TPL candidate. Alternatively, module decoupling algorithms analyze internal dependencies and structural relationships within the application to segment the codebase into distinct modules, facilitating the identification of library instances.

Once potential TPLs are isolated, the **feature extraction stage** focuses on gathering distinguishing characteristics that uniquely represent each candidate. This typically involves extracting structural and behavioral features such as Android API calls, control flow graphs (CFGs), and method signatures. Android API calls provide insight into the functionality

of the library, CFGs capture execution logic, and method signatures facilitate structural comparison. The extracted features are then used to generate a set of feature vectors, capturing the unique characteristics and patterns of each module. These feature vectors serve as the basis for the final stage of the pipeline, library identification.

Android APIs play a crucial role in understanding the functionality and services utilized by TPLs. By analyzing the specific API calls made by a library, we can infer its external interactions and dependencies, which aids in determining its purpose and behavior. The usage patterns of these APIs provide valuable insights into how a TPL interacts with the Android system and other components within the application.

Control Flow Graphs (CFGs) illustrate the logical execution paths within a library, depicting how different parts of the code interact. By capturing execution flows, CFGs offer insight into the internal structure and operational behavior of the TPL. This graphical representation is particularly useful for identifying unique control patterns and distinguishing between different libraries with similar functionalities.

Method signatures provide a detailed characterization of the methods used within a TPL, including method names, parameter types, and return values. By examining method signatures, it becomes possible to identify specific functionalities and their implementations. This level of granularity is particularly valuable in distinguishing between libraries that may share similar API usage patterns but differ in implementation details.

In the **library identification stage**, the final step of the pipeline, the extracted feature vectors are analyzed to confirm and identify specific TPLs within an application. This is

Table 1 Permissions required by each payment library, used to identify their presence in the binaries

Payment Library	Permission(s)
Google Play API	com.android.billingclient
PayPal	com.paypal.android, com.paypal.sdk
Amazon	com.amazon.inapp.purchasing
Braintree	com.braintreepayments.api
Stripe	com.stripe.android.view
AliPay	com.alipay.sdk
UrbanAirship	com.urbanairship
Samsung	com.samsung.android.sdk.iap
RevenueCat	com.revenuecat.purchases, com.revenuecat.purchases.api

achieved using either clustering-based identification or similarity comparison-based identification.

The **clustering-based approach** first filters out the primary module—typically the core application logic—using heuristics such as package names, code size, and distinguishing features associated with the host application. The remaining non-primary modules are then grouped based on feature similarity. These clusters are subsequently compared against a reference database of known TPLs. If the feature vector of a cluster matches an entry in the database, the corresponding module can be classified as a TPL.

In contrast, the **similarity comparison-based approach** treats all modules within an application as potential TPL candidates. This method first requires the collection of known TPL samples. The extracted features of these collected libraries are then directly compared against the features of the application modules to identify matches, thereby determining the presence of specific TPLs within the application.

4.2 PayScan Architecture

In Figure 2, we give a high-level overview of the PayScan workflow, which involves detection, security analysis, and reporting. To identify embedded payment SDKs, PayScan employs a two-step static analysis approach: (i) Manifest-based detection: Identifying payment libraries based on the permissions and components declared in the *AndroidManifest.xml* file. (ii) Byte-pattern scanning: Extracting library-specific byte sequences and API signatures from the decompiled source code to detect SDKs even when obfuscation is applied.

4.2.1 Manifest-Based Detection

The *AndroidManifest.xml* contains, among other things, the list of permissions. When an app uses a third-party library, it must include the permissions of that library in its manifest file. The PayScan tool already decodes most of the files in each APK, including the *AndroidManifest.xml* file. There-

Table 2 Package names related to payment libraries observed in the *AndroidManifest.xml* from sample .apk

Payment library	Package name
Kin	com.kin.ecosystem
bitcoinj	org.bitcoinj
RobotMediaBilling	net.robotmedia.billing
Stellar	org.stellar.sdk
Authorize.net	net.authorize.acceptsdk.network
Fortumo	com.fortumo.android
PayAnywhere	com.nabancard.payanywheresdk
CardinalCommerce	com.cardinalcommerce
SquareUp	com.squareup.ui

fore, the decompiled file, stored in a byte array, is searched for the payment library's specific permissions. Though, searching for specific permissions with the tag of "<user-permission>" worked for some of the payment libraries as shown in Table 1.

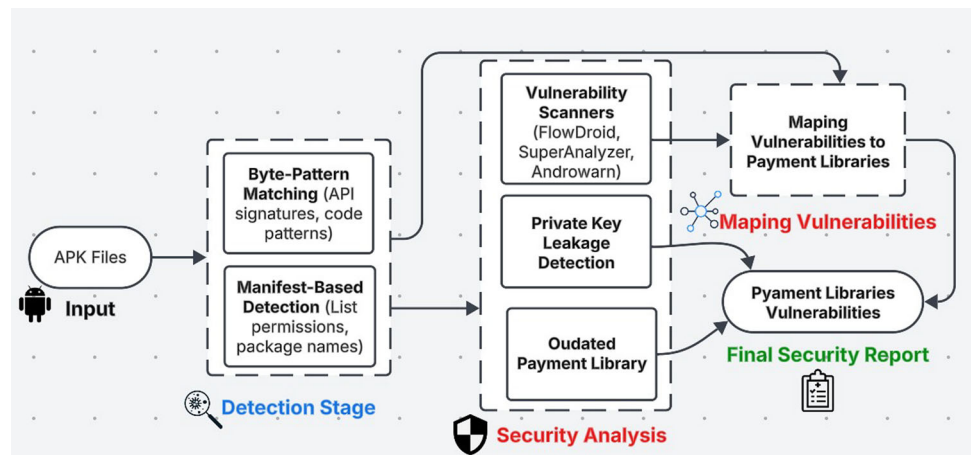
However, some libraries did not request any permissions and were included in the *AndroidManifest.xml* as third-party packages with various tags such as "<android-package>" and "<application>."

Detecting the presence of these libraries is primarily achieved by searching for their package names within the application or identifying external resources they reference. The package names associated with these libraries are listed in Table 2. However, in some cases, library developers intentionally obscure or omit direct references to their libraries within the application files to prevent reverse engineering or mitigate potential exploitation by attackers. Consequently, an alternative detection approach is required to handle such cases effectively.

4.2.2 Byte-Pattern Matching in Source Code

For libraries that do not explicitly declare their presence in the manifest, PayScan performs byte-pattern analysis to iden-

Fig. 2 A high-level overview of the PayScan workflow, which entails detecting the presence of payment libraries and then identifying their security vulnerabilities



tify unique code signatures within an application's compiled source files. This ensures detection even when (i) the library is indirectly embedded within another SDK, (ii) the manifest file does not explicitly reference the library, or (iii) the application is obfuscated to conceal third-party components.

To detect payment libraries at the bytecode level, PayScan follows these steps: (i) Extracts DEX files from the APK and converts them into a readable format. (ii) Searches for library-specific byte patterns and API calls unique to each payment SDK. (iii) Validates the findings by examining associated logging statements, package names, and network requests.

We manually analyzed the source code of open-source payment libraries and reverse-engineered sample applications using Dex2jar and Apktool to extract byte-pattern signatures. Table 3 lists a selection of unique strings used for detection.

By combining manifest scanning with byte-pattern detection, PayScan achieves high accuracy in identifying embedded payment SDKs, even in obfuscated applications.

4.3 Supported Payment Libraries

Our current implementation supports 18 popular payment libraries. The popularity was determined based on AppBrain [32], an intelligence tailored towards providing detailed information concerning Android apps and libraries. We ended up with the following list of libraries: *PayPal*, *Google Play API*, *Stripe*, *AliPay*, *Urban Airship*, *Samsung In-App Purchase*, *Amazon in-app purchasing*, *RevenueCat*, *Braintree*, *Kin*, *Bitcoinj*, *RobotMediaBilling*, *Stellar*, *Authorize.net*, *Fortumo*, *PayAnywhere*, *CardinalCommerce*, *SquareUp*. We have excluded Android Vending from our analysis due to its deprecated status and the challenges in accurately detecting its usage. Our findings indicate that third-party libraries, such as Facebook Analytics for Billing, often initialize Android Vending even when the application itself does not actively use it. This leads to significant false positives, making it difficult

to distinguish genuine usage from indirect library dependencies. Given these limitations, we focus only on actively maintained and explicitly integrated payment libraries in our analysis.

4.4 In-house Security Checks

In addition to the security features offered by the integrated security scanners, we will discuss them in detail in the following sections, our tool implements two additional security features. First, it detects private key leakage and thus whether or not keys are stored securely. Second, it detects whether an app is using outdated payment libraries.

4.4.1 Detection of Outdated Payment Libraries

To assess whether applications were using outdated payment libraries, we implemented a version extraction module. If a detected payment library had a known version identification pattern, we extracted its version string and compared it against the latest version found in our dataset. The five selected libraries (Amazon IAP, Braintree, Samsung Pay, Stripe, and Urban Airship) were chosen because their versioning information was consistently retrievable. Libraries whose versions were difficult to detect due to heavy obfuscation or inconsistent implementation were excluded from this analysis.

4.4.2 Detection of Private Key Leaks

To detect private key leaks in Android applications, we implemented a static analysis approach using pattern-based string matching and entropy analysis. Our methodology consists of the following steps:

1. Extracting Application Code and Resources: We first decompile each APK file using reverse engineering tools

Table 3 Examples of byte-pattern signatures used to detect payment libraries

Payment Library	Unique Byte Pattern
PayPal	"paypal.sdk", "com.paypal.android"
Amazon	"Amazon Appstore required", "com.amazon.inapp.purchasing"
Braintree	"BraintreePaymentActivity", "com.braintreepayments.api"
Stripe	"https://stripe.com/docs/stripe.js", "X-Stripe-Client-User-Agent"
AliPay	"com.alipay.android.app.pay.IAlipayCallback", "com.alipay.android"
Samsung	"com.samsung.android.sdk.samsungpay", "Samsung Pay Service"

such as Apktool and Dex2jar to extract readable code and resources from compiled DEX files.

2. Searching for Hardcoded Keys Using Pattern-Based Matching: We applied predefined regex-based patterns to identify potential cryptographic keys embedded in the application code. These patterns were designed to match common formats of cryptographic secrets, including:

- RSA private keys (PEM format)
- API keys and access tokens
- AES encryption keys (128-bit and 256-bit formats)
- Base64-encoded secrets

3. Applying Entropy-Based Filtering: Since not all strings that match the above patterns are valid cryptographic keys, we applied Shannon entropy analysis to filter out false positives. Genuine cryptographic keys typically exhibit high entropy due to their randomness, whereas regular strings have lower entropy.

4. Identifying Key References and Context: For each detected key, we examined its surrounding code to determine:

- Whether the key is part of a hardcoded configuration file or logging statement.
- If it is associated with a known SDK or library (e.g., Google SDK, Netty).
- Whether the key appears in an obfuscated or non-obfuscated context.

For obfuscated cases, we used heuristics to match detected keys against known patterns in non-obfuscated applications.

5. Classifying Keys as Test or Production: To assess whether leaked keys belonged to test environments or production deployments, we compared them against:

- Commonly known public test keys from SDK documentation.
- Keys found in sample applications.
- Repeated usage of the same key across multiple applications.

Keys that were identical to well-known test credentials were classified as test keys. However, keys with no known references were flagged for further investigation.

6. Reporting and Validation: Detected private keys were stored in a structured report format, logging their file locations, entropy scores, and possible SDK associations. We manually validated a sample of detected keys to verify our approach's effectiveness.

This methodology enables automated large-scale detection of leaked private keys while reducing false positives through entropy-based filtering. Future work could integrate dynamic analysis to assess whether detected keys are actively used during application runtime.

4.5 Security Scanners

To identify security vulnerabilities in payment libraries, we employed existing security analysis tools to scan APKs and then parsed their output for relevant findings. Given the variety of available tools, we evaluated multiple options based on their effectiveness in detecting vulnerabilities specific to third-party libraries, their compatibility with our processing pipeline, and their precision in reporting meaningful security issues.

Our methodology follows a structured approach: first, we identify the presence of payment libraries in an APK, then scan the APK for vulnerabilities, and finally parse the reports to isolate security issues related specifically to these libraries. Therefore, a crucial requirement for selecting a security scanner was its ability to generate reports that allow us to trace detected vulnerabilities back to the corresponding third-party library. Tools that did not support this level of granularity-i.e., those that only provided general vulnerability reports without distinguishing between host app and third-party library issues-were excluded from consideration.

After this evaluation, we selected the following tools for our analysis: FlowDroid [33], SuperAnalyzer [35], and Androwarn [34]. These tools collectively provide robust coverage of security vulnerabilities, including static taint analysis, permission misuse detection, and heuristic-based malware identification, while also enabling us to correlate discovered vulnerabilities with specific payment libraries.

To integrate these scanners into our workflow, we developed dedicated parsers for each tool to extract and standardize vulnerability reports. While our current approach focuses on these three tools, our framework is designed to accommodate additional security scanners in future iterations, enabling broader analysis as new tools and techniques become available.

FlowDroid is a static analysis tool that, as the name suggests, can follow an Android app's data flow to find any possible data leaks. The output of running FlowDroid is an *.xml* file that lists the leaks found and their method(s). The crucial part is that these methods are listed by their full library path, which facilitates finding data leaks related to the payment libraries. *SuperAnalyzer* scans APK for vulnerabilities and generates easily navigable *.html* and *.json* reports. It divides the detected vulnerabilities into five categories: critical, major, medium, low, and warning. The tool provides extensive descriptions for the vulnerabilities, including the origin, thus, making it possible to tie them to the payment libraries. Finally, *Androwarn* generates three types of reports and offers three types of output files: *.txt*, *.json*, and *.html*. Each report contains a series of vulnerability categories. Under each of these categories, a series of issues, in addition to the faulty line of code, is depicted, enabling filtering of the vulnerabilities based on the package names of the payment libraries.

4.6 Implementation

Our tool, PayScan, is implemented in C++ for efficient static analysis of Android applications. The system consists of three main phases: identification, security scanning, and vulnerability parsing.

Identification Phase: The tool scans the application's DEX and manifest files to detect the presence of payment libraries. It applies byte-pattern matching techniques to identify SDK-specific signatures, ensuring that even obfuscated instances can be recognized. **Security Scanning Phase:** Once a payment library is identified, the APK is analyzed using multiple security scanners (Androwarn, SuperAnalyzer, and FlowDroid) to detect potential vulnerabilities. The scanning process includes taint analysis, permission misuse detection, and cryptographic flaw identification. Additionally, PayScan checks if the payment library is outdated and checks for leaked cryptographic keys. **Vulnerability Parsing Phase:** The results from security scanners are parsed and filtered to isolate issues directly related to the detected payment libraries. The tool categorizes vulnerabilities based on severity levels (critical, high, medium, low) and correlates them with specific SDK versions.

The system is designed for extensibility, allowing new payment libraries to be added by updating signature definitions and feature extraction modules. Our detection approach

is similar to prior byte-pattern-based analysis techniques, but it is tailored specifically for third-party payment SDKs, making it the first tool designed for large-scale security assessment of mobile payment libraries.

4.7 Validation and Refinement

To validate and refine our implementation, we conducted an iterative testing phase using two distinct samples of applications. This phase aimed to assess detection accuracy, identify potential false positives and false negatives, and fine-tune our approach accordingly.

The first sample set consisted of 43 applications known to contain at least one of the 18 supported payment libraries. To collect this dataset, we leveraged AppBrain [32] and 42matters [30], both of which provide detailed insights into library usage within Android applications. We then retrieved the corresponding APKs from APKMirror [31]. This dataset served as a ground truth for evaluating our tool's ability to correctly identify payment libraries.

The second sample set comprised 40 applications that our tool initially classified as not containing any payment libraries. These applications were selected to assess the potential rate of false negatives and ensure that our detection approach did not mistakenly overlook libraries due to obfuscation or structural variations.

For both sample sets, we performed manual reverse engineering and source code inspection to verify the correctness of our tool's classification. This involved decompiling the applications, analyzing package structures, and examining key indicators of payment library usage. Insights gained from this validation process led to refinements in byte pattern selection and improvements in our detection heuristics, ultimately enhancing the accuracy and robustness of our framework.

5 Experiments and Results

In this section, we evaluate our tool's effectiveness in three key aspects: (i) validating our detection approach, (ii) analyzing the current security state of payment libraries in Android applications, and (iii) comparing our tool's detection accuracy, efficiency, and resilience to obfuscation against LibScout and LibRadar.

5.1 Data Set

To assess the accuracy and efficiency of the new tool, we utilized a large dataset of 10,553 Android application (APK) files. This dataset was collected using a crawler for *APKMirror*, a well-known online archive hosting various Android application versions [2]. The applications were selected

Table 4 The usage distribution of 18 payment libraries in the 10,553 apps dataset

Payment Library	Frequency	Percentage
AliPay	111	1.1%
Amazon IAP	306	2.9%
Authorize.net	2	0.0 %
bitcoinj	12	0.1%
Braintree	102	1.0%
Cardinal Commerce	75	0.7%
RevenueCat	60	0.6%
Fortumo	3	0.0%
Google Play Billing	1900	18.0%
kin	3	0.0%
PayAnywhere	0	0.0%
PayPal	148	1.4%
RobotMedia	16	0.2%
Samsung Pay	90	0.9%
Square	0	0.0%
Stellar	8	0.1%
Stripe	107	1.0%
Urban Airship	165	1.6%

across diverse categories and functionalities to ensure a comprehensive analysis of the Android application ecosystem. Categories were assigned based on their classification in the Google Play Store, as listed on APKMirror. The most represented categories in the dataset were Finance, Communication, Productivity, Entertainment, and Games.

5.2 Analysis of payment library usage

The distribution of payment libraries among the data set of 10,553 Android applications provides important insights into the adoption of in-app payments, as well as the popularity of individual libraries. The distribution of payment libraries is presented in Table 4.

The table shows that Google Play Billing is the most popular in-app payment library used throughout the dataset, which is somehow expected. First, Android is mainly developed and maintained by Google. Second, Google provides rich documentation and support to make the integration of Google Play Billing smooth and easy. After Google Play Billing comes Amazon IAP and then PayPal. Both being of course large names in the industry. Another interesting finding from this analysis is that some apps integrate more than one payment library. For instance, one app we found to use 6 payment libraries, 4 apps use 5 payment libraries, 96 apps use 4 payment libraries, 276 apps use 3 payment libraries and 702 apps use 2 payment libraries. Finally, out of 10553 Android appli-

cations, 3108 apps uses at least one payment library. This comes down to 29.45% of the applications.

5.3 Outdated SDK Versions

To evaluate whether developers use outdated payment SDKs, we analyzed the versions of five widely used payment libraries: Amazon IAP, Braintree, Samsung Pay, Stripe, and Urban Airship. These libraries were selected based on two main criteria. First, their version numbers were consistently extractable from APK files through structured version strings or byte-pattern analysis. Second, they are among the most commonly used payment SDKs in Android applications, making them relevant for assessing the prevalence of outdated software.

The process of identifying outdated SDKs involved two main steps. First, we extracted version information from each application that integrated one of these five payment libraries. The method of extraction varied depending on the SDK; for example, some libraries explicitly stored version information in metadata files or logging messages, while others required a pattern-based analysis of compiled bytecode. If no version information was found, the case was classified as *Unknown*. Second, we determined whether an extracted version was outdated by comparing it against the most recent version found in our dataset. This approach provides a reasonable approximation of outdated software usage, though it does not guarantee that the absolute latest release at the time of analysis was detected. If an application used an SDK version older than the most recent found in our dataset, it was classified as *outdated*.

Our findings, summarized in Table 5, reveal that a significant proportion of applications rely on outdated SDK versions. Out of 770 applications where a version number was identified, 552 were running an outdated version, representing **71.7%** of the analyzed applications. This percentage is considerably higher than anticipated, indicating that many developers do not regularly update their payment libraries. It is important to note that an outdated SDK version does not necessarily imply a security vulnerability. The risk depends on how old the version is and whether it contains known security flaws. However, maintaining up-to-date payment libraries is critical for security and compliance with industry standards such as the PCI-DSS (Payment Card Industry Data Security Standard).

During this analysis, we encountered cases where version information could not be retrieved, which are labeled as "Unknown" in Table 5. This was primarily due to obfuscation techniques applied to some applications, as well as cases where the payment SDK was present in the application but its version string was not easily accessible. Overall, these findings emphasize the need for continuous monitoring of third-party SDKs to ensure that applications are

Table 5 The number of applications using outdated payment libraries. The analysis considered only five payment libraries

Payment Provider	Analyzed Apps	Unknown	Outdated	Latest Version
Amazon IAP	306	95	194	2.0.76.4
Braintree	102	1	94	4.15.0
Samsung Pay	90	7	85	2.9
Stripe	107	48	103	20.11.0
Urban Airship	165	68	76	16.7.2

running secure and compliant versions of payment libraries. Future work could expand this analysis to include additional payment SDKs and investigate the security implications of specific outdated versions.

5.4 Key Leakage

Our analysis identified **397 private key leaks** across **212 applications**, meaning that approximately **2.0% of the analyzed applications contained at least one leaked private key**. As discussed in Section 4, private key leakage can have serious security consequences, including unauthorized decryption of sensitive data or compromise of authentication mechanisms.

To assess the potential impact, we examined whether these leaked keys were production keys or merely introduced for testing purposes. Out of the 397 private keys detected, only 21 had direct references in the source code, making it possible to determine their context. Most references were found within obfuscated methods, making it difficult to discern their exact function. In cases where deobfuscation was feasible, we found that several leaked keys were associated with the Google SDK and Netty, an NIO client-server framework. These appeared to be test keys rather than production secrets. For obfuscated methods, we attempted to match leaked keys to their non-obfuscated equivalents to determine their usage context.

Additionally, we identified private key leaks related to TunnelBear VPN and AliPay. However, the specific security risk posed by these cases remains unclear, as it is difficult to determine what data these private keys are used to encrypt or sign. We suspect that TunnelBear may use a static private key to decrypt traffic, but further analysis would be required to confirm this.

While many of the leaked keys may be test credentials, they still pose a security risk if mistakenly reused in production environments or if an attacker can exploit them to gain unauthorized access. This highlights the importance of auditing applications for hardcoded secrets, particularly in obfuscated environments where detection is more challenging.

5.5 Vulnerability Detection Results

In this section, we present our findings from scanning the dataset using three security scanners: Androwarn, SuperAnalyzer, and FlowDroid. The objective was to identify security vulnerabilities and privacy risks associated with the 18 payment libraries detected in our dataset of 10,553 Android applications. To ensure relevance, only vulnerabilities that could be directly associated with these libraries were considered, filtering out host app issues.

5.5.1 Androwarn

Androwarn [34] produces a structured output file containing details on detected security issues, including affected components and the source of the vulnerability within the .apk file. Table 6 summarizes the vulnerabilities identified in payment libraries by Androwarn.

The *Device Settings Harvesting* vulnerability category includes logs that may expose sensitive data, such as debugging information, internal variables, or device identifiers. *Code Execution* refers to cases where the payment library interacts with native code, which could introduce security risks if improperly handled.

5.5.2 SuperAnalyzer

SuperAnalyzer identified 20,478 vulnerabilities across the dataset. Table 7 categorizes vulnerabilities among the payment libraries based on severity levels.

The key findings from SuperAnalyzer are as follows: (i) Critical vulnerabilities were detected in AliPay and Fortumo, including SSL misconfigurations, WebView XSS, and insecure data transmission. (ii) Weak cryptographic algorithms were found in Amazon, Stripe, bitcoinj, and Fortumo, indicating a potential risk of data interception. (iii) Several libraries, including Google Play API, Samsung, and Robot-MediaBilling, were flagged for exported components, which could be exploited by malicious apps to access sensitive functions. (iv) Logging issues (unchecked logs, URL disclosure) were common across multiple libraries, potentially exposing internal implementation details.

Table 6 Vulnerabilities detected in payment libraries using Androwarn [34]

Library	Vulnerability
Google Play API	Device Settings Harvesting
PayPal	Device Settings Harvesting
Amazon	Code Execution, Device Settings Harvesting
Stripe	Device Settings Harvesting
AliPay	Device Settings Harvesting
Samsung	Device Settings Harvesting
Kin	Device Settings Harvesting
PayAnywhere	Device Settings Harvesting
CardinalCommerce	Device Settings Harvesting

Table 7 Vulnerabilities detected in payment libraries using SuperAnalyzer [35], categorized by severity

Library	Vulnerability	Severity
Google Play API	Exported activity, Unchecked Logs, Internet permission, Exported receiver	Medium
PayPal	Unchecked Logs, URL Disclosure, Rooted Device Detection	Low
Amazon	URL Disclosure, Weak Algorithms, Unchecked Logs	High
Braintree	Rooted Device Detection, URL Disclosure	Medium
Stripe	Weak Algorithms, Unchecked Logs, Email Disclosure, URL Disclosure	High
AliPay	Accepting all SSL certificates, WebView ignores SSL errors, URL Disclosure	Critical
Samsung	Exported activity, Unchecked Logs, URL Disclosure	Medium
Kin	Unchecked Logs, URL Disclosure	Low
bitcoinj	Temp File Use, URL Disclosure, Weak Algorithms	High
RobotMediaBilling	Exported receiver, Unchecked Logs, Exported service, Exported activity	High
Fortumo	WebView XSS, Weak Algorithms, Unchecked Logs, Get SIM Operator, Sending SMS/MMS, URL Disclosure	Critical
CardinalCommerce	Rooted Device Detection, IP Disclosure, Unchecked Logs, URL Disclosure	Medium
SquareUp	Rooted Device Detection, Superuser Privileges, Unchecked Logs, URL Disclosure	High

Table 8 Comparison of the three security scanners used in our study

Tool	Focus Area	Total Vulnerabilities	Most Critical Issues
Androwarn	Static security analysis	–	Device settings harvesting, Code execution risks
SuperAnalyzer	Comprehensive vulnerability scanning	20,478	WebView XSS, Weak encryption, Exported components
FlowDroid	Data flow analysis	Low	Device ID, SQL cache leaks

Table 9 Comparison of detection accuracy, precision, and recall

Tool	Recall (%)	Precision (%)	Accuracy (%)
LibScout	79.43	31.32	54.92
LibRadar	11.41	73.04	29.12
PayScan (Ours)	49.29	53.33	48.65

Table 10 Comparison of execution time

Tool	Mean Execution Time (seconds)
LibScout	6.05
LibRadar	288.64
PayScan (Ours)	1.41

These findings indicate that all analyzed payment libraries exhibit security or privacy concerns, highlighting the need for ongoing vulnerability assessment and mitigation. (see table 9, 10, 11)

5.5.3 FlowDroid

FlowDroid reported significantly fewer vulnerabilities compared to the previous tools, as it focuses on data leakage analysis. The tool identified leaks primarily related to device identifiers and sensitive data storage. The most notable leaks detected include: (i) AliPay: Leakage of Mac Address, SSID, Device ID, and Subscriber ID. (ii) Amazon: Leakage of HTTP URLs, Cached Credentials, Logs in two applications. (iii) One application was found to leak a SQL cache, which could provide access to an internal database file.

These results suggest that FlowDroid excels in detecting privacy-related leaks, whereas SuperAnalyzer and Androwarn provide broader security assessments.

5.5.4 Cross-Tool Comparison and Implications

Table 8 summarizes the key differences in vulnerability detection among the three tools.

The disparity in results highlights the importance of multi-tool analysis, as each scanner focuses on different aspects of security. SuperAnalyzer detects the most vulnerabilities, providing a broad assessment of security weaknesses. Androwarn focuses on static analysis, identifying hardcoded risks and native code execution. Finally, FlowDroid specializes in detecting privacy leaks, uncovering cases where sensitive information is exposed.

By integrating findings from all three tools, we provide a comprehensive evaluation of security risks in payment libraries.

5.6 Comparison Study

In this section, we compare PayScan against LibScout and LibRadar, two widely used third-party library (TPL) detection tools. The comparison is based on three key metrics: (i) Accuracy and Precision – How effectively each tool identifies payment libraries, (ii) Efficiency – The time required to analyze an application, and (iii) Resilience to Obfuscation – How well each tool detects libraries in obfuscated APKs. To ensure fairness, we evaluate all tools on a dataset of 385 Android applications, selected using Cochran's sampling method to represent a diverse set of real-world apps. The dataset includes both non-obfuscated and obfuscated versions of the same apps, with obfuscation applied using Allatori, ProGuard, DexProtector, and Stringer.

5.6.1 Accuracy and Precision

Accuracy measures how well a tool correctly detects third-party payment libraries, while precision indicates how many of the reported detections are actually correct. We define:

$$\text{Accuracy} = \frac{\text{Total Correct}}{\text{Total Correct} + \text{Total Missed} + \text{Total False Positives}}$$

$$\text{Precision} = \frac{\text{Total Correct}}{\text{Total Correct} + \text{Total False Positives}}$$

$$\text{Recall} = \frac{\text{Total Correct}}{\text{Total Libraries}}$$

The results show that LibScout has the highest recall (79.43%) but the lowest precision (31.32%), indicating a high false positive rate. LibRadar, on the other hand, achieves high precision (73.04%) but suffers from extremely low recall (11.41%), meaning it often fails to detect embedded payment libraries. PayScan strikes a balance between recall (49.29%) and precision (53.33%), making it **more reliable in reducing false positives than LibScout while detecting more libraries than LibRadar. This balance is essential in practical applications where minimizing incorrect detections is as important as maximizing recall.

5.6.2 Efficiency

We measure the average execution time of each tool on our dataset.

PayScan significantly outperforms both LibScout and LibRadar in execution time. It is $4.3\times$ faster than LibScout, which suffers from Java heap overflows in large applications, and $204\times$ faster than LibRadar, which relies on outdated analysis techniques that slow down execution. PayScan's speed advantage comes from its optimized byte-pattern matching algorithm, which processes DEX and manifest files in a single-pass analysis, reducing memory overhead.

5.6.3 Resilience to Obfuscation

To evaluate obfuscation resilience, we apply four obfuscation techniques to 295 APKs and compare detection rates before and after obfuscation.

LibScout drops from 89% to 14% under DexGuard, showing severe degradation under advanced obfuscation. LibRadar drops from 65% to 9%, performing poorly under all obfuscation levels. PayScan initially achieves 95% accuracy but drops to 10% under DexGuard, similar to LibScout. While all tested tools, including our approach, performed poorly against DexGuard-obfuscated applications, this limitation should be viewed in context. Prior research [29] analyzing over 1.7 million Google Play apps found that only 25% of developers use any obfuscation, with the majority relying on the weaker ProGuard rather than advanced tools like DexGuard. This suggests that, while DexGuard remains a challenge, it is not widely adopted, reducing its practical impact on real-world security evaluations.

5.6.4 Summary of Findings

PayScan achieves the best balance between recall and precision, outperforming LibScout and LibRadar in the recall. It is four times ($4\times$) faster than LibScout and two hundred times ($200\times$) faster than LibRadar, making it the most efficient detection tool. Finally, PayScan performs similarly to LibScout under obfuscation, but improvements are needed for advanced techniques like DexGuard. While LibScout detects more libraries, it suffers from false positives. LibRadar has high precision but low recall. PayScan provides a well-balanced trade-off, making it the most practical solution for large-scale payment library detection.

Developer Recommendations

Based on our study results and conclusions, we recommend:

1. **Update SDKs regularly** - Over 70% of apps used outdated libraries, increasing vulnerability risks.
2. **Avoid hardcoded secrets** - We detected 397 private key leaks. Store secrets securely using Android Keystore or encrypted storage.
3. **Conduct static analysis pre-release** - Tools like PayScan, FlowDroid, or SuperAnalyzer can catch issues early.
4. **Monitor obfuscation impacts** - Obfuscation can hinder both functionality and security analysis. Use it carefully and validate SDK behavior post-obfuscation.

6 Conclusion & Future Directions

In this paper, we investigated the security risks associated with third-party payment libraries in Android applications. We presented *PayScan*, a detection and analysis tool that combines manifest and byte-pattern techniques with three static security scanners. Applying *PayScan* to 10,553 Android apps, we identified 18 unique payment libraries and revealed multiple critical security concerns, including:

- A high prevalence of outdated SDKs (71.7% of analyzed apps), potentially increasing exposure to known vulnerabilities.
- 397 detected private key leaks across 212 apps, with evidence suggesting both test and production usage.
- Over 20,000 security issues discovered by static analysis, including SSL misconfigurations, WebView XSS, and weak cryptographic usage.

Comparative evaluation with *LibScout* and *LibRadar* shows that *PayScan* offers a strong trade-off between recall and precision while outperforming both in runtime efficiency. While *PayScan* remains effective against common obfuscation techniques, we observed diminished performance under advanced tools like DexGuard (recall drops to 10%).

Limitations: Our approach currently relies on static analysis and cannot detect runtime-only behaviors, dynamic code loading, or encrypted payloads. Moreover, obfuscation methods such as DexGuard still pose detection challenges. These are noted in our results and reflected in our roadmap.

Future Work: We plan to (i) extend *PayScan*'s detection engine to handle dynamic code loading and native binaries, (ii) integrate dynamic analysis (e.g., sandboxing and runtime

Table 11 Obfuscation resilience results across different tools

Tool	No Obfuscation	Allatori	ProGuard	DexGuard
LibScout	89%	27%	41%	14%
LibRadar	65%	22%	30%	9%
PayScan (Ours)	95%	35%	50%	10%

taint tracking), and (iii) develop and release an open-source version of PayScan with extended functionality to support ongoing research and security auditing of mobile payment ecosystems.

Author Contributions Fadi Mohsen: Conceptualization, methodology design, tool development oversight, manuscript writing, and overall project coordination. Manar Alohal: Writing, idea development, and result interpretation. Usman Rauf: Writing, idea development, and result interpretation. Dominic Therattil: Conducted the comparison analysis between our solution and two other tools, writing. Loran Oosterhaven: Tool development and implementation, application scanning, writing, and security vulnerability detection. All authors reviewed, revised, and approved the final manuscript and agree to be accountable for the integrity and accuracy of the work.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Al-Hakimi, A.M.H., BakarMdSultan, A., AbdulGhani, A.A., Ali, N.M., Admodisastro, N.I.: Hybrid obfuscation technique to protect source code from prohibited software reverse engineering. *IEEE Access* **8**, 187326–187342 (2020)
- APKMirror. APKMirror: Free and Safe Android APK Downloads, 2023. Accessed: July, 2023]
- Backes, M., Bugiel, V., Derr, E.: Reliable third-party library detection in android and its security applications. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, page 356–367, New York, NY, USA, (2016). Association for Computing Machinery
- Backes, M., Bugiel, V., Derr, E.: Reliable third-party library detection in android and its security applications. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, page 356–367, New York, NY, USA, (2016). Association for Computing Machinery
- Bläsing, T., Batyuk, L., Schmidt, A.-D., Camtepe, S.A., Albayrak, S.: An android application sandbox system for suspicious software detection. In *2010 5th International Conference on Malicious and Unwanted Software* 55–62 (2010)
- Cho, T., Kim, H., Yi, J.H.: Security assessment of code obfuscation based on dynamic monitoring in android things. *IEEE Access* **5**, 6361–6371 (2017)
- Darvish, H., Husain, M.: Security analysis of mobile money applications on android. In *2018 IEEE International Conference on Big Data (Big Data)*, pages 3072–3078, (2018)
- Degenhard, J.: Smartphone users in the world 2013–2028
- Demetriou, S., Merrill, W., Yang, W., Zhang, A., Gunter, C.A.: Free for all! assessing user data exposure to advertising libraries on android. In *Proceedings 2016 Network and Distributed System Security Symposium*, San Diego, CA, (2016). Internet Society
- Derr, E., Bugiel, S., Fahl, S., Acar, Y., Backes, M.: Keep me updated: An empirical study of third-party library updatability on android. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, page 2187–2200, New York, NY, USA, (2017). Association for Computing Machinery
- Flynn, L., Klieber, W.: Smartphone security. *IEEE Pervasive Computing*, 14(4), (2015)
- Geethanjali, D., Ying, T.L., Melissa, C.W.J., Balachandran, V.: Aeon: Android encryption based obfuscation. In *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy*, CODASPY '18, page 146–148, New York, NY, USA, (2018). Association for Computing Machinery
- Grace, M.C., Zhou, W., Jiang, X., Sadeghi, A.R.: Unsafe exposure analysis of mobile in-app advertisements. In *Proceedings of the Fifth ACM Conference on Security and Privacy in Wireless and Mobile Networks*, WISEC '12, page 101–112, New York, NY, USA, (2012). Association for Computing Machinery
- Guo, Y., Gou, Y., Chen, W., Zhang, M., Sun, D.: Defect detection of third party payment processes in android applications. In *2019 IEEE International Conference on Computation, Communication and Engineering (ICCCCE)*, pages 67–70, (2019)
- Khandelwal, A., Mohapatra, A.K.: An insight into the security issues and their solutions for android phones. In *2015 2nd International Conference on Computing for Sustainable Global Development (INDIACom)*, pages 106–109, (2015)
- Li, M., Wang, P., Wang, W., Wang, S., Dinghao, W., Liu, J., Xue, R., Huo, W., Zou, W.: Large-scale third-party library detection in android markets. *IEEE Trans. Software Eng.* **46**(9), 981–1003 (2020)
- Li, M., Wang, W., Wang, P., Wang, S., Wu, D., Liu, J., Xue, R., Huo, W.: Libd: Scalable and precise third-party library detection in android markets. In *Proceedings - 2017 IEEE/ACM 39th International Conference on Software Engineering, ICSE 2017*, Proceedings - 2017 IEEE/ACM 39th International Conference on Software Engineering, ICSE 2017, pages 335–346, United States, July (2017). Institute of Electrical and Electronics Engineers Inc. Funding Information: ACKNOWLEDGMENTS We thank anonymous reviewers for their invaluable comments and suggestions. This research was supported in part by the National

- Science Foundation of China (No. 61572481, 61402471, 61472414 and 61602470), the Program of Beijing Municipal Science and Technology Commission (No. Y6C0021116), the US National Science Foundation (Grant No. CCF-1320605), and Office of Naval Research (Grant No. N00014-13-1-0175, N00014-16-1-2265, and N00014-16-1-2912). Publisher Copyright: 2017 IEEE.; 39th IEEE/ACM International Conference on Software Engineering, ICSE 2017 ; Conference date: 20-05-2017 Through 28-05-2017
18. Liu, B., Liu, B., Jin, H., Govindan, R.: Efficient privilege de-escalation for ad libraries in mobile apps. In *Proceedings of the 13th Annual International Conference on Mobile Systems, Applications, and Services, MobiSys '15*, page 89–103, New York, NY, USA, (2015). Association for Computing Machinery
 19. Ben Lutkevich. What is obfuscation?, 4 2021
 20. Ma, Z., Wang, H., Guo, Y., Chen, X.: Libradar: Fast and accurate detection of third-party libraries in android apps. *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*, pages 653–656, (2016)
 21. Ma, Z., Wang, H., Guo, Y., Chen, X.: Libradar: Fast and accurate detection of third-party libraries in android apps. In *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*, pages 653–656, (2016)
 22. Moonsamy, V., Batten, L.: Android applications: Data leaks via advertising libraries. In *2014 International Symposium on Information Theory and its Applications*, pages 314–317, (2014)
 23. Mos, A., Chowdhury, MM.: *Mobile Security: A Look into Android*, pages 638–642. 2020 IEEE International Conference on Electro Information Technology (EIT). IEEE, (2020)
 24. Narayanan, A., Chen, L., Chan, CK.: Addetect: Automated detection of android ad libraries using semantic analysis. In *2014 IEEE Ninth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)*, pages 1–6, (2014)
 25. Short, A., Li, F.: Android smartphone third party advertising library data leak analysis. In *2014 IEEE 11th International Conference on Mobile Ad Hoc and Sensor Systems*, pages 749–754, (2014)
 26. Statista. Mobile wallet transactions worldwide, 2025. Accessed: February 10, 2025
 27. Traynor, P., Butler, K., Bowers, J., Reaves, B.: Fintechsec: Addressing the security challenges of digital financial services. *IEEE Secur. Priv.* **15**(5), 85–89 (2017)
 28. Ulqinaku, E., Stefa, J., Mei, A.: Scan-and-pay on android is dangerous. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 1–6, (2019)
 29. Wermke, D., Huaman, N., Acar, Y., Reaves, B., Traynor, P., Fahl, S.: A large scale investigation of obfuscation use in google play. In *Proceedings of the 34th Annual Computer Security Applications Conference, ACSAC '18*, page 222–235, New York, NY, USA, (2018). Association for Computing Machinery
 30. 42matters: Mobile sdk explorer. <https://42matters.com/sdks>. (visited on 08/29/2022)
 31. Apkmirror: Download apks. <https://www.apkmirror.com/>. (visited on 08/29/2022)
 32. Appbrain: Android billing libraries. <https://www.appbrain.com/stats/libraries/tag/billing/android-billing-libraries>. (visited on 02/17/2022)
 33. Flowdroid: Static data flow tracker. <https://github.com/google/FlowDroid>. (visited on 08/29/2022)
 34. Androwarn: Platform for android application analysis. <https://github.com/androgard/androgard>. (visited on 08/29/2022)
 35. Superanalyzer: Secure, unified, powerful and extensible rust android analyzer. <https://superanalyzer.rocks/>. (visited on 08/29/2022)
 36. Zhan, X., Fan, L., Liu, T., Chen, S., Li, L., Wang, H., Xu, Y., Luo, X., Liu, Y.: Automated third-party library detection for android applications: are we there yet? In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, ASE '20*, page 919–930, New York, NY, USA, (2021). Association for Computing Machinery
 37. Zhan, X., Liu, T., Fan, L., Li, L., Chen, S., Luo, X., Liu, Y.: Research on third-party libraries in android apps: A taxonomy and comprehensive survey. *IEEE Transactions on Software Engineering*, pages 1–32, September (2021)
 38. Zhang, Z., Diao, W., Hu, C., Guo, S., Zuo, C., Li, L.: An empirical study of potentially malicious third-party libraries in android apps. In *Proceedings of the 13th ACM Conference on Security and Privacy in Wireless and Mobile Networks, WiSec '20*, page 144–154, New York, NY, USA, (2020). Association for Computing Machinery

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.